Every chapter so far assumes access to a scalar reward signal: dynamic programming requires $r(s, a)$, model-free RL observes r_t after each transition, and the Bellman optimality equation presupposes that rewards are known or observable. When they are neither, the DP/RL machinery cannot be applied directly.

In many domains, scalar rewards are unavailable but ordinal preferences over trajectories are easy to elicit. A human evaluator cannot assign a meaningful numerical score to a paragraph of text, but can reliably say "response A is better than response B." The raw data is a set of trajectory pairs with binary preference labels. RLHF uses these ordinal comparisons to learn a proxy reward function r_θ, which then serves as the scalar signal for standard RL optimization. This is not an inverse problem in the IRL sense; the goal is not to rationalize observed behavior, but rather a two-stage forward problem in which the analyst first learns a reward from human judgments and then solves the resulting MDP. **?** demonstrated that this approach could train agents without an explicit reward function. RLHF has since become the predominant method for aligning large language models.

0.1 Learning Rewards from Preferences

The canonical RLHF framework is built on a formal model of human preference. The observed data consists of tuples (s, y_w, y_l), where s is a context, and y_w and y_l are two outputs, with y_w being the "winner" preferred by a human. Assuming preferences follow a latent utility model, the Bradley-Terry model (**?**) gives the probability that y_w is preferred: $P(y_w \succ y_l|s) = \sigma(r_\theta(s, y_w) - r_\theta(s, y_l))$, where $r_\theta : \mathcal{S} \times \mathcal{Y} \to \mathbb{R}$ is a learned *reward model* parameterized by θ, trained to approximate human preferences (not the ground-truth reward, which is unobserved), and $\sigma(\cdot)$ is the logistic function. This formulation is a binary logit model (Section **??**, Equation **??**).[1] The reward model parameters θ are estimated by minimizing the negative log-likelihood of the observed human choices:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(r_\theta(s, y_w) - r_\theta(s, y_l)\right)\right], \tag{1}$$

[1] **?** discuss the contrasts and synergies between machine learning and structural econometrics, including the shared reliance on logit-based choice models that underlies both RLHF and dynamic discrete choice estimation.

In the LLM setting, the "outputs" y_w and y_l are token sequences, i.e., trajectories of the autoregressive policy[2], so preferences are over trajectories rather than single actions. The preference loss in Equation (1) is MLE of a choice model where the "alternatives" are trajectory segments and the "choice" is the human-preferred one.

0.2 The RLHF Pipeline and Direct Optimization

The learned r_θ then serves as a proxy objective for policy optimization. Building on the reward-learning and RL fine-tuning framework of **?** and **?**, the canonical three-stage pipeline was formalized by **?**. First, a base pretrained model is initialized via supervised fine-tuning (SFT)[3] on a small dataset of high-quality demonstrations, yielding an initial policy π^{SFT}. This grounds the model in the desired style and format. The second step is the reward model training as described, using preference data generated from this π^{SFT}.

In the third step, the SFT policy π_ϕ is fine-tuned via PPO (Section **??**) to maximize the frozen reward model r_θ, with a KL-divergence penalty preventing the policy from drifting into regions where r_θ is unreliable. The objective is

$$J(\phi) = \mathbb{E}_{s\sim\mathcal{D}, y\sim\pi_\phi(\cdot|s)}[r_\theta(s,y)] - \lambda_{KL}\mathbb{E}_{s\sim\mathcal{D}}[D_{KL}(\pi_\phi(\cdot|s)\,||\,\pi^{SFT}(\cdot|s))], \tag{2}$$

where D_{KL} is the Kullback-Leibler divergence and λ_{KL} controls the penalty strength.[4] Without this constraint, the policy exploits inaccuracies in r_θ to achieve high proxy scores with degenerate behavior ("reward hacking"), the RLHF analogue of divergence under function approximation (Section **??**).

The KL-regularized objective in Equation (2) admits a Bayesian interpretation (**?**). In this view, the reference policy π^{SFT} acts as a prior distribution over plausible responses. The reward model r_θ provides evidence, specifying which responses are more desirable. The goal of alignment is to find the posterior distribution π^* that optimally combines the

[2] An *autoregressive* language model generates text token by token: at each step it outputs a distribution over the vocabulary conditioned on all preceding tokens, then samples the next token; the full response is therefore a trajectory in token space and the model acts as a sequential policy over a vocabulary-sized action set.

[3] *Pretraining* optimizes a language model to predict the next token across a massive text corpus, producing broad linguistic knowledge with no behavioral objective. *Supervised fine-tuning* (SFT) continues training on a small curated dataset of (prompt, ideal-response) pairs to specialize the model toward the desired task and establish the reference policy π^{SFT} from which the KL penalty is measured.

[4] λ_{KL} denotes the KL penalty weight, reserving β for model parameters and γ for discount factors. The standard RLHF literature, including **?**, uses β for this parameter.

prior with this evidence. This ideal posterior policy takes the form of Equation (3):

$$\pi^*(y|s) \propto \pi^{SFT}(y|s) \exp\left(\frac{r_\theta(s,y)}{\lambda_{KL}}\right). \tag{3}$$

The reward function scaled by λ_{KL} defines the log-likelihood, so the KL-regularized objective $J(\phi)$ is equivalent (up to an additive constant) to the Evidence Lower Bound (ELBO) for this Bayesian inference problem. Maximizing the RLHF objective via PPO is therefore variational inference: finding the policy π_ϕ that minimizes KL divergence to π^*. This reframes the KL penalty as a structural component of the inference problem rather than an ad-hoc regularizer.

Despite this closed-form characterization, the three-stage RLHF pipeline is complex to implement, requiring training multiple large models and a computationally expensive RL loop. Direct Preference Optimization (DPO), introduced by **?**, collapses the pipeline into a single supervised learning objective by reparameterizing the reward function in terms of π^* and π^{SFT}, as in Equation (4):

$$r(s,y) = \lambda_{KL} \log\left(\frac{\pi^*(y|s)}{\pi^{SFT}(y|s)}\right) + \lambda_{KL} \log Z(s). \tag{4}$$

When this analytical expression for the reward is substituted into the Bradley-Terry preference loss from Equation (1), the unknown partition function $Z(s)$ cancels out. This yields a loss function that depends only on the policy π_ϕ being optimized and the fixed reference policy π^{SFT}. The DPO objective, given in Equation (5), is thus a simple binary cross-entropy loss over policy likelihoods:

$$\mathcal{L}_{DPO}(\phi; \pi^{SFT}) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\lambda_{KL}\log\frac{\pi_\phi(y_w|s)}{\pi^{SFT}(y_w|s)} - \lambda_{KL}\log\frac{\pi_\phi(y_l|s)}{\pi^{SFT}(y_l|s)}\right)\right]. \tag{5}$$

This objective is optimized on ϕ using standard supervised learning with a static preference dataset. The gradient increases the likelihood of preferred responses y_w and decreases the likelihood of dispreferred responses y_l, relative to the reference policy.

0.3 Recent Developments

A subsequent innovation leverages DPO to create an iterative self-improvement loop, reducing reliance on static, human-annotated data (**?**). In this paradigm, dubbed Self-

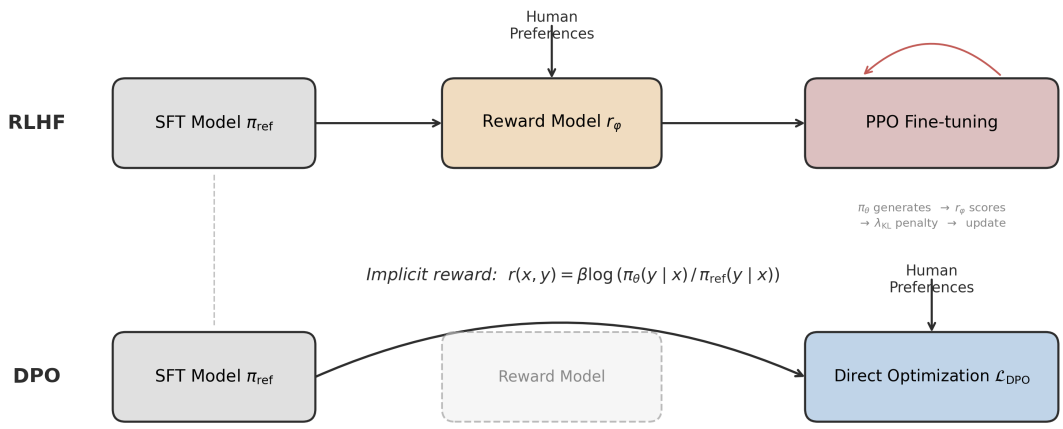

Figure 1: RLHF versus DPO pipelines. Top row: the three-stage RLHF pipeline trains a reward model from human preferences, then uses PPO to fine-tune the policy with a KL penalty. Bottom row: DPO collapses the pipeline into a single supervised learning objective over preference pairs, eliminating the explicit reward model (ghosted box).

Rewarding Language Models, a single language model acts as both the instruction-following agent and its own reward model. The process begins with a base model fine-tuned to have both instruction-following and evaluation capabilities ("LLM-as-a-Judge"). In each subsequent iteration, the current model generates a new preference dataset for itself by producing multiple responses to a set of prompts and then evaluating its own outputs to assign scores. These scores are used to construct preference pairs (y_w, y_l), which form an AI-generated feedback dataset. The model is then fine-tuned on this new data using the DPO loss. This iterative process creates a feedback loop where enhancements in instruction-following ability lead to better reward modeling, which in turn fuels the next round of policy optimization.

While RLHF and its successors have proven effective at aligning model behavior with human preferences, several open issues remain. The framework does not aim to solve the identification problem in the strict econometric sense; the learned reward model r_θ (whether explicit or implicit in DPO) is a proxy for preference, not necessarily the uniquely identified "true" utility function required for welfare analysis.[5] Furthermore,

[5]The identification problem in preference learning mirrors that in discrete choice. The reward function is identified only up to an additive constant and requires a location normalization.

the notion of "human feedback" is a significant simplification, as the preferences being optimized are those of a small, non-representative group of paid labelers, raising important questions about whose values are being embedded in these systems. Finally, the fine-tuning process can lead to degraded performance on standard academic benchmarks, a phenomenon called the *alignment tax* (**?**).[6] Mitigating these challenges while scaling alignment beyond the limits of human data collection remains a key frontier for the field.

0.4 Simulation Study: Preference Learning in Job Search

A worker searches for jobs in a labor market with compensating differentials, following a McCall (1970)-style search model. Each job is characterized by a wage $w \in \{20, 28, 38, 50, 65, 82, 100, 125\}$ (thousands) and an amenity level $z \in \{0, 1, \ldots, 6\}$ capturing commute quality, flexibility, and job security. The state space has 112 states: 56 searching states in which the worker observes a pending offer (w_i, z_j) and decides to accept or reject, plus 56 employed states (w_i, z_j) in which the worker decides to stay or quit. The offer distribution exhibits compensating differentials, with wage rank and amenity rank negatively correlated ($\rho = -0.74$): high-wage offers cluster with low amenities and vice versa. The worker's true per-period utility is $u(w, z) = \alpha \log(w) + (1 - \alpha)z$ with $\alpha = 0.6$, but this function is unobserved; the worker can only compare career trajectories ("I prefer path A to path B"), exactly as in stated-preference surveys in labor economics. While searching, the worker receives the unemployment benefit $u_b = \alpha \log(b)$ where $b = 28$. Layoffs occur with probability $p = 0.05$ per period, and the discount factor is $\gamma = 0.95$. Dynamic programming gives $V^*(s_0) = 74.13$; the optimal policy accepts 25 of 56 offer types and stays employed at 25 of 56 job types. Preference data is generated by rolling out a uniform random policy from a random searching state. Each rollout produces a career segment of $L = 15$ periods recording states and actions. For each of K comparisons, two independent career segments are generated; the segment with higher cumulative discounted utility under the true (unobserved) utility function is labeled as preferred via the Bradley-Terry model. Figure 2 shows the optimal accept/reject and stay/quit boundaries.

Six methods are compared across $K \in \{25, 50, 100, 200, 500, 1{,}000, 2{,}000, 5{,}000\}$, aver-

[6] The alignment tax, as described by **?**, refers to performance regressions on public NLP benchmarks (SQuAD, DROP, HellaSwag) that result from RLHF fine-tuning.

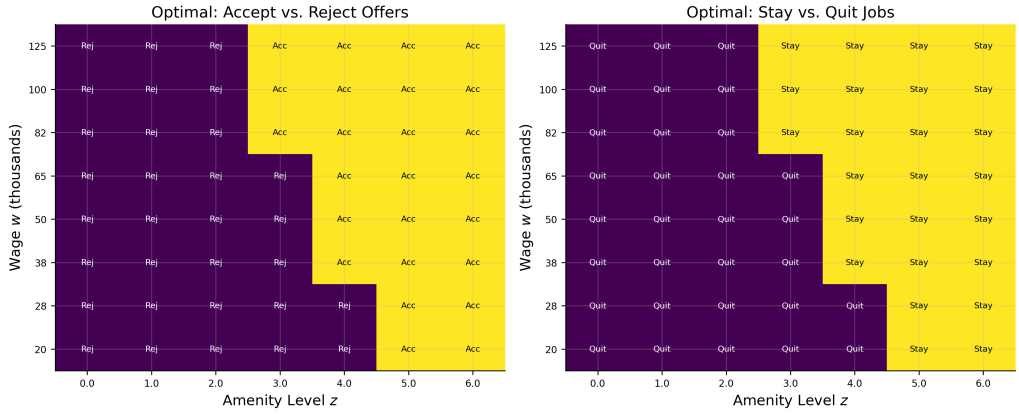

Figure 2: Left: the optimal accept/reject boundary for searching states in wage-amenity space. Right: the optimal stay/quit boundary for employed states. Both boundaries cut diagonally, reflecting the tradeoff between wage and amenity quality under compensating differentials.

aged over 30 seeds. The neural network RLHF reward model is a two-layer MLP trained by Bradley-Terry MLE on the K segment pairs; per-transition rewards are discount-weighted and summed to obtain a segment score, and the resulting 112-state reward table is solved by value iteration.[7] The correctly specified structural model parameterizes utility as $\hat{u} = \hat{\alpha}\log(w) + (1-\hat{\alpha})z$, estimating the single parameter $\hat{\alpha}$ via Bradley-Terry MLE, then solves the MDP. The misspecified model uses $\hat{u} = \hat{\alpha}\log(w) + (1-\hat{\alpha})\bar{z}$, where $\bar{z} = 3$ is the mean amenity level; this model ignores amenity variation across jobs, treating all amenities as identical. DPO trains a tabular softmax policy directly from trajectory comparisons, bypassing reward modeling entirely.[8] Tabular Q-learning (10,000 episodes, $\varepsilon = 0.15$, learning rate 0.1) and exact DP provide scalar-reward baselines. All four preference methods receive identical comparison data per seed; DPO receives a same-state variant generated from the same seed.

Figure 3 reports the main results. Three findings stand out. First, the correctly specified structural model dominates: even at $K = 25$ it achieves 99.9% of DP-optimal, illustrating the power of correct specification when the model has a single free parameter. Second, the neural network converges more slowly but reaches 99.9% by $K = 5{,}000$, reflecting the higher sample complexity of a flexible model relative to a one-parameter

[7] The neural network has 4 inputs (normalized log-wage, normalized amenity, employment indicator, action), 32 hidden units per layer, and ∼1,200 parameters. The logistic loss from Equation (1) is applied to discount-weighted segment reward sums.

[8] DPO uses 112 logit parameters ϕ_s, one per state, trained via the DPO loss (Equation 5) with Adam optimization over a sweep of $\lambda_{KL} \in \{0.01, 0.05, 0.1, 0.5, 1.0, 5.0\}$, selecting the λ_{KL} that minimizes training loss. The reference policy is uniform: $\pi^{SFT}(a|s) = 0.5$. To match the LLM setup where both completions condition on the same prompt, DPO comparison pairs start from the same initial state.

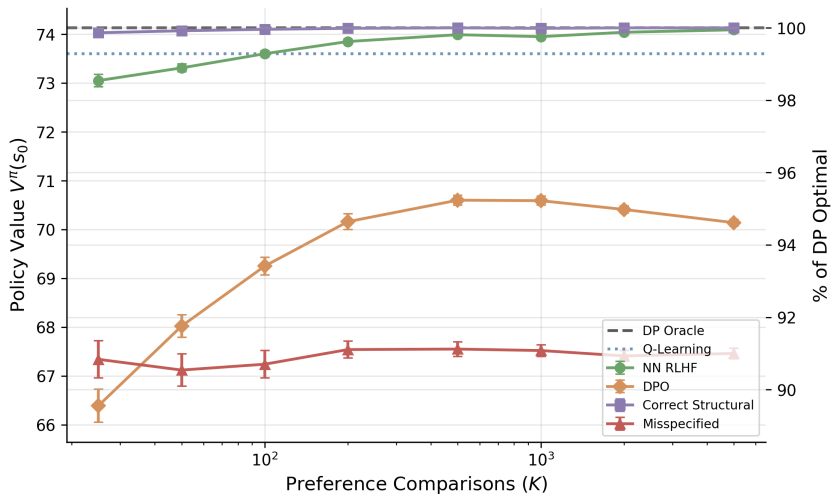

Figure 3: Policy value $V^\pi(s_0)$ versus number of preference comparisons K for all six methods (30 seeds, $L = 15$). The right axis shows the percentage of DP-optimal value.

structural specification. Third, DPO plateaus at approximately 95% by $K = 500$ and does not improve further.[9] This plateau is qualitatively different from the gridworld failure (−118%): DPO recovers a reasonable policy, but the gap does not close with additional data.[10] The misspecified constant-amenity model plateaus at 91% regardless of K, because additional preference data cannot correct the omitted variable.

Table 1: Diagnostics at $K = 5{,}000$ (single seed): policy agreement with π^*, value-function correlation, and mean accepted wage and amenity for each method.

Method	Policy agree. (%)	V^π corr.	Mean amenity	Mean wage	$\hat{\alpha}$
NN RLHF	96.4	1.000	4.67	73	—
DPO	57.1	0.777	3.38	63	—
Correct	100.0	1.000	4.84	71	0.597
Misspecified	50.0	0.889	3.00	70	—
Optimal	100.0	1.000	4.84	71	—

Table 1 provides state-level diagnostics at $K = 5{,}000$. The structural model and neural network achieve near-perfect policy agreement with π^* (100% and 96% respectively). DPO agrees on only 57% of states with value-function correlation 0.78, systematically underselecting on both wage and amenity dimensions.[11] The misspecified model agrees

[9] DPO learns only from (s, a) pairs visited in training trajectories generated by the random behavioral policy. It cannot propagate value to undervisited states the way value iteration does after learning a reward model, so states poorly covered by the behavioral policy remain suboptimal regardless of K.

[10] DPO fails catastrophically in gridworld because transitions are stochastic (10% slip probability) and rewards are transition-dependent. The same (s, a) pair yields different rewards depending on whether the agent slipped, so the DPO loss conflates policy quality with transition luck. In the job search model, accept/reject deterministically changes employment status, and only the 5% layoff probability introduces stochastic transitions.

[11] DPO's mean accepted amenity of 3.4 parallels the misspecified structural model's 3.0, though the mechanisms

on 50%, with disagreements concentrated where amenity variation matters.[12]

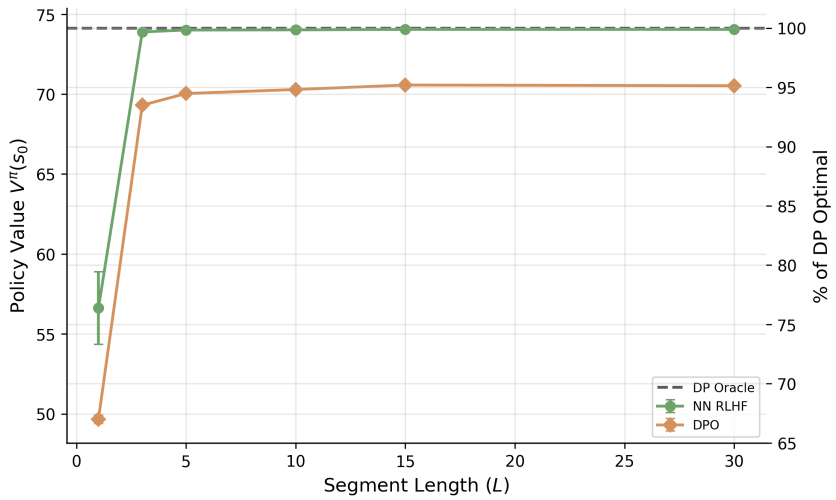

Figure 4: Policy value $V^{\pi}(s_0)$ versus segment length L at $K = 2{,}000$ for the neural network and DPO (20 seeds). The right axis shows the percentage of DP-optimal value.

Figure 4 reports the segment length ablation at $K = 2{,}000$. At $L = 1$, both methods perform poorly because single-step comparisons carry minimal information about long-run value. The neural network recovers rapidly (by $L = 3$) because the reward model aggregates per-transition estimates over longer segments and value iteration propagates them through the full transition structure; DPO improves monotonically but plateaus at its ∼95% ceiling, as it must learn the policy directly from trajectory comparisons without access to the transition model.

Two-stage RLHF separates preference estimation (a static econometric problem) from dynamic programming (which exploits the known transition model); DPO conflates the two and forfeits the transition structure that economists typically have access to.[13]

differ: the misspecified model ignores amenity variation by construction, while DPO underweights amenities because the random behavioral policy underrepresents high-amenity employed states in the training data.

[12] An online versus offline ablation for the neural network at $K = 1{,}000$ (20 seeds) shows comparable performance: online 73.92 ± 0.05, offline 73.99 ± 0.03 ($p = 0.09$). The random behavioral policy already provides diverse career trajectories covering the full wage-amenity space.

[13] This advantage is specific to settings where the transition model is known or estimable; in domains without a tractable transition model, DPO's single-stage approach avoids compounding errors from reward model estimation.